

Flutter は不要な更新をスキップ	パフォーマンスを向上させ, 処理を最適化する.
----------------------------	-------------------------

g. Flutter のウィジェット階層 vs Java (Android 開発)

- 主要な違い:

特徴	Flutter (Dart)	Android (Java)
UI の定義	レイアウトとロジックが Dart 内で完結	UI は XML, ロジックは Java/Kotlin
ウィジェット構造	すべてがウィジェット (ネスト構造)	ビューは別オブジェクトとして ID で管理
状態管理	setState() で宣言的に管理	リスナーで手動更新

第一級オブジェクトとしての関数と引数としての使用

a. 「第一級オブジェクト」とはどういう意味か？

- Dart では,関数は「第一級オブジェクト」として扱われ,他の変数と同じように扱うことができる.
- 関数を変数に代入したり,引数として渡したり,別の関数から返したりできる.
- これにより,コードの柔軟性と再利用性が向上する.

b. 第一級クラス関数の主な使い方

- Dart の関数の主な使い方

特徴	説明	例
変数に代入	関数を変数として保存できる.	<code>var fn = sayHello;</code>
引数として渡す	関数を別の関数に渡せる.	<code>executeFunction(sayHello);</code>
関数を返す	関数から別の関数を返すことができる.	<code>Function getFunction() { return sayHello; }</code>

c. 例：関数を引数として渡す

```
void sayHello() {
  print('Hello!');
}

void executeFunction(void Function() fn) {
  fn(); // Call the function passed as an argument
}

void main() {
  executeFunction(sayHello); // Pass the sayHello function
}
```

- 説明:

- ◇ sayHello() 関数を executeFunction() に渡す.
- ◇ executeFunction() の中で,渡された関数 fn() を実行.
- ◇ これにより,関数を柔軟に実行できる.

d. 例：引数付きの関数

```
void showMessage(String message) {
    print(message);
}

void performAction(void Function(String) action, String
message) {
    action(message); // Call the action with the message
}

void main() {
    performAction(showMessage, 'Button was clicked!'); //
Pass the showMessage function
}
```

- 説明:
 - ◇ showMessage(String message) はメッセージを表示する関数.
 - ◇ performAction() は関数 (action) と message を受け取る.
 - ◇ performAction() の中で,渡された関数がメッセージと共に実行される.

e. なぜ Flutter においてこれは有用なのか？

- 関数を引数として渡すことは,Flutter でのユーザーインタラクション処理に特に便利.
- 例えば,Flutter のボタン (ElevatedButton) は,onPressed プロパティで関数を受け取る.

f. 応用例：関数を戻り値として返す

```
Function createMultiplier(int factor) {
    return (int number) => number * factor;
}

void main() {
    var doubleIt = createMultiplier(2); // Returns a function
that multiplies by 2
    print(doubleIt(5)); // Output: 10
}
```

- 説明:
 - ◇ createMultiplier(int factor) は factor で掛け算する関数を返す.
 - ◇ createMultiplier(2) を呼び出すと,2 倍する関数が返される.
 - ◇ doubleIt(5) を実行すると 10 になる.

g. Dart と Java の関数の違い

- 主な違い:

特徴	Dart (Flutter)	Java (Android)
関数の扱い	関数は第一級オブジェクト	関数はクラス内のメソッド
関数の引数渡し	簡単に渡せる	インターフェースや匿名クラスが必要
関数の返却	関数を返せる	インターフェースやラムダ式を利用

三項条件操作

a. 三項演算子（条件演算子）とは？

- 三項条件とは,if-else 文を 1 行で書くためのショートカットである.
- 条件をチェックし,true または false に応じて処理を決定する.
- シンプルな条件を扱う場合,コードを短く,読みやすくできる.

b. 基本構文

```
condition ? valueIfTrue : valueIfFalse;
```

- 構文の説明:

- ◇ condition → チェックする条件 (true または false).
- ◇ ? → condition が true の場合,この後の値が返される.
- ◇ : → condition が false の場合,コロンの後の値が返される.

c. 例: if-else 文を三項演算子に書き換える

Regular if-else (通常の if-else)	Ternary Condition (三項条件演算子)
<pre>if (isLoggedIn) { return Text('Welcome back!'); } else { return Text('Please log in!'); }</pre>	<pre>Text(isLoggedIn ? 'Welcome back!' : 'Please log in!');</pre>

- 説明:

- ◇ if-else を複数行で書く必要がなくなる.
- ◇ isLoggedIn に基づいて適切な値を直接返す.

d. 三項演算子を使うべき場面は？

- 三項条件を使うべきとき:
 - ◇ 条件がシンプルで,2つの値のどちらかを選ぶ場合.
 - ◇ 短い表現で素早く判断したい場合.
 - ◇ UI ウィジェット (Text, Button など) でコードを簡潔にしたい場合.